

TRABALHO PRATICO - FASE III

Loja Online

Relatorio Tecnico do Aplicativo

Arquitetura: MVC + DAO em Java, com interface web local

Persistencia: arquivos binarios com cabecalho e exclusao logica por lapide

Evolucao: relacionamento N:N entre Pedido e Produto e consulta ordenada com Arvore B+

Esta fase transforma os itens de pedido em uma tabela associativa persistida, com chave primaria composta (idPedido, idProduto) . O sistema oferece navegacao nos dois sentidos da associacao e exibe o catalogo em ordem diretamente pelas folhas de uma Arvore B+ persistente.

1. Escopo implementado

- Tabela associativa real `PedidoProduto` para o relacionamento `Pedido N:N Produto`.
- PK composta formada exclusivamente pelas FKs `idPedido` e `idProduto`.
- Consultas web `Pedido -> Produtos` e `Produto -> Pedidos`.
- Árvores B+ persistentes para a associação e para a listagem ordenada de produtos.
- Integridade na criação, atualização e remoção, incluindo ajuste de estoque.
- Migração automática dos itens salvos no formato legado das fases anteriores.

2. Modelo de dados

Entidade	Campos relevantes	Papel
Pedido	<code>id</code> , <code>idCliente</code> , <code>idCupom</code> , <code>dataPedido</code> , <code>valorTotal</code>	Tabela principal de vendas
Produto	<code>id</code> , <code>nome</code> , <code>preco</code> , <code>estoque</code>	Tabela principal do catálogo
PedidoProduto	<code>idPedido</code> PK/FK, <code>idProduto</code> PK/FK, <code>quantidade</code>	Tabela associativa N:N

```
PEDIDO 1 ----- N PEDIDO_PRODUTO N ----- 1 PRODUTO
                    PK (idPedido, idProduto)
```

A classe `Pedido` ainda consegue ler vetores legados para migração, mas novos itens e todas as consultas desta fase utilizam `PedidoProduto`.

3. Persistência e índices

3.1 Arquivo associativo

```
data/pedido_produto.db
[cabecalho int: total de insercoes]
[lapide boolean][tamanho int][idPedido int][idProduto int][quantidade int] ...
```

O arquivo novo segue o padrão das tabelas anteriores: cabeçalho de quatro bytes, lapide booleana, tamanho e payload binário. Exclusões não removem bytes fisicamente; elas marcam a lapide e os índices são reconstruídos somente com registros ativos.

3.2 Estrutura B+

Índice persistido	Chave	Aplicação
-------------------	-------	-----------

pedido_produto.db.pedido.bplus.db	(idPedido, idProduto)	Listar produtos de um pedido
pedido_produto.db.produto.bplus.db	(idProduto, idPedido)	Listar pedidos de um produto
produtos.db.ordem_id.bplus.db	idProduto	Listar catalogo em ordem

BPlusTreeIndex persiste nos e folhas encadeadas. A pagina /produtos percorre as folhas do indice para apresentar IDs crescentes, sem chamada de ordenacao em memoria principal.

O hash extensivel preexistente continua sendo usado nas pesquisas pontuais por PK e em Cliente -> Pedidos , pois e apropriado quando a operacao principal e obter uma chave exata.

4. Integração com a interface e o CRUD

Operação	Tratamento aplicado
Criar pedido	Valida cliente, produtos, quantidades e estoque; grava itens em <code>PedidoProduto</code> .
Atualizar pedido	Le itens associativos ativos, recalcula estoque e total, marca itens antigos com lapide e grava os novos.
Excluir pedido	Aplica lapide aos itens associados e devolve quantidades ao estoque.
Excluir produto	Bloqueia a exclusão caso exista item ativo referenciando o produto.
Consultar N:N	Rotas web consultam B+ por pedido ou pela ordem reversa por produto.

A aplicação não utiliza console como interface. A demonstração é executada em `http://localhost:18080`, especialmente nas telas `/produtos` e `/pedidos`.

5. Formulário técnico obrigatório

1. Qual foi o relacionamento N:N escolhido e quais tabelas ele conecta?

Foi escolhido **Pedido N:N Produto**. A tabela intermediária persistida `PedidoProduto` conecta as tabelas principais `Pedido` e `Produto`.

2. Qual estrutura de índice foi utilizada (B+ ou Hash Extensível)? Justifique.

Na nova tabela foi utilizada a **Árvore B+**, porque pares com o mesmo pedido ou produto ocupam intervalos contíguos e podem ser recuperados eficientemente. As folhas encadeadas também permitem a consulta ordenada obrigatória do catálogo sem ordenar resultados em memória. O hash extensível anterior foi preservado para consultas exatas por ID.

3. Como foi implementada a chave composta da tabela intermediária?

A chave é o par `(idPedido, idProduto)`, sem ID artificial. Para a B+, cada par é codificado em um `long`: 32 bits para o primeiro ID e 32 bits para o segundo. O DAO impede a repetição de um par ativo.

4. Como é feita a busca eficiente de registros por meio do índice?

A consulta por pedido percorre o intervalo `(idPedido, 0)` a `(idPedido, Integer.MAX_VALUE)`. O índice reverso realiza a mesma operação para produto. O catálogo ordenado percorre as folhas ligadas da B+ de produtos.

5. Como o sistema trata a integridade referencial entre as tabelas?

Pedidos somente aceitam produtos existentes e com estoque. Atualizações substituem associações com lapide e ajustam estoque. A remoção de pedido remove logicamente seus itens e recompoe

estoque; a remocao de produto referenciado e recusada.

6. Como foi organizada a persistencia da nova tabela?

`pedido_produto.db` possui cabecalho `int` e registros no formato `lapide + tamanho + payload`. Seus arquivos B+ separados sao reconstruidos a partir dos itens ativos ao iniciar a aplicacao.

7. Como o codigo da tabela intermediaria se integra com o CRUD?

`PedidoController` utiliza `PedidoProdutoDAO` na criacao, atualizacao, consulta e exclusao de pedidos. `ProdutoController` consulta a tabela para impedir exclusao invalida. `Main.App` expoe as consultas no navegador.

8. Como esta organizada a estrutura de diretorios e modulos?

Diretorio	Conteudo atual
<code>Model/</code>	Entidades, incluindo <code>PedidoProduto.java</code> .
<code>DAO/</code>	Persistencia, <code>PedidoProdutoDAO.java</code> , <code>BPlusTreeIndex.java</code> e hash existente.
<code>Controller/</code>	Regras de integridade, estoque e operacoes N:N.
<code>Main/</code> e <code>View/</code>	Servidor HTTP, rotas, formularios e estilo web.
<code>data/</code>	Dados e indices binarios persistidos na execucao.
<code>docs/</code>	Documentacao tecnica e este PDF.

6. Compilacao e demonstracao

```
New-Item -ItemType Directory -Force out\classes | Out-Null
$sources = Get-ChildItem -Recurse -Filter *.java -Path Controller,DAO,Main,Model,Util,View |
ForEach-Object { $_.FullName }
javac --release 8 -d out\classes $sources
java -cp out\classes Main.App
```

Demonstracao: criar um pedido com produtos em `/pedidos`; consultar os produtos do pedido e os pedidos de um produto; acessar `/produtos` para visualizar a ordem produzida pela Arvore B+.

7. Links da entrega

GitHub: <https://github.com/lucaszanfa/Tp-Aedes3>

Video explicativo da Fase I: https://www.youtube.com/watch?v=o_7LNxR3HZc

Video explicativo da Fase II: <https://www.youtube.com/watch?v=3b0R70VgKEw>

Video explicativo da Fase III: link a ser inserido pelo grupo apos gravacao e publicacao.

8. Conclusao

A Fase III implementa efetivamente a associacao antes conceitual entre pedidos e produtos. A solucao possui chave primaria composta, persistencia no padrao do projeto, navegacao N:N bidirecional, integridade referencial no CRUD e uma aplicacao pratica de Arvore B+ para recuperacao ordenada em disco.

Documento preparado para a entrega da Fase III. O unico item dependente de acao externa e o URL do video novo, que deve substituir o campo indicado apos sua publicacao.